



各种类型的线程池

北京理工大学计算机学院
金旭亮

一个用于测试线程池的任务类

为了方便展示线程池的工作方式，设计了一个Task类，此类的实例将作为一个工作任务被提交给线程池执行。

```
public class Task implements Runnable {  
    4 usages  
    private String _info = "";  
    3 usages  
    public Task() {  
    }  
    public Task(String info) { _info = info; }  
    @Override  
    public void run() {  
        try {  
            Thread.sleep( millis: 500);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        var threadName :String = Thread.currentThread().getName();  
        if (_info.length()== 0) {  
            _info="执行线程任务";  
        }  
        System.out.println(threadName + _info + " " + LocalTime.now());  
    }  
}
```

FixedThreadPoolTest

```
public class FixedThreadPoolTest {  
    public static void main(String[] args) {  
        //创建一个固定包容4个线程的线程池  
        ExecutorService executorService = Executors.newFixedThreadPool( nThreads:4);  
        for (int i = 0; i < 1000; i++) {  
            executorService.execute(new Task());  
        }  
    }  
}
```

Run: FixedThreadPoolTest x

"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"

pool-1-thread-1执行线程任务 19:26:38.684640700
pool-1-thread-2执行线程任务 19:26:38.684640700
pool-1-thread-3执行线程任务 19:26:38.684640700
pool-1-thread-4执行线程任务 19:26:38.684640700
pool-1-thread-1执行线程任务 19:26:39.195162100
pool-1-thread-2执行线程任务 19:26:39.195162100
pool-1-thread-3执行线程任务 19:26:39.195162100

不管任务再多，始终都是固定数目线程在执行任务。

查看newFixedThreadPool的源码.....

```
public static ExecutorService newFixedThreadPool(int nThreads) {  
    return new ThreadPoolExecutor(nThreads, nThreads,  
        keepAliveTime: 0L, TimeUnit.MILLISECONDS,  
        new LinkedBlockingQueue<Runnable>());  
}
```

可以看到，这种类型的线程池，拥有一个没有容量上限的任务队列，这样一来，当请求数越来越多并且无法及时处理完毕时，会造成大量的内存占用，可能会导致OOM（内存溢出错误）。



使用newSingleThreadExecutor()创建的单一线程池，它的任务队列类型也是一样的，同样可能造成OOM。

人为引发OOM错误



```
public class FixedThreadPoolOOM {  
    1 usage  
    private static ExecutorService executorService =  
        Executors.newFixedThreadPool( nThreads:1);  
    public static void main(String[] args) {  
        for (int i = 0; i < Integer.MAX_VALUE; i++) {  
            executorService.execute(new SubThread());  
        }  
    }  
}
```

当线程处理耗费很长的时间，而任务的添加又非常快速时，将会导致内存耗尽的情况发生.....

```
class SubThread implements Runnable {  
    @Override  
    public void run() {  
        try {  
            System.out.println(Thread.currentThread().getName()+"正在执行");  
            //休眠很长的时间  
            Thread.sleep( millis: 10000000000);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

运行截图

设定JVM使用8M内存，这会很快耗尽.....

Name:

Run on: Local machine [Manage targets...](#)

Run configurations may be executed locally or on a target: for example in a Docker Container or on a remote host using SSH.

Build and run

java 17 Oracle OpenJDK -Xmx8m -Xms8m



Run: fixed.FixedThreadPoolOOM x

```
"C:\Program Files\Java\jdk-17\bin\java.exe" -Xmx8m -Xms8m "-javaagent:C:\Program Files\JetBrains\IntelliJ  
pool-1-thread-1正在执行
```

```
Exception: java.lang.OutOfMemoryError thrown from the UncaughtExceptionHandler in thread "main"
```

```
Process finished with exit code 1
```

单骑仗剑走天涯.....



```
public class SingleThreadExecutor {  
    public static void main(String[] args) {  
        ExecutorService executorService = Executors.newSingleThreadExecutor();  
        for (int i = 0; i < 1000; i++) {  
            executorService.execute(new Task());  
        }  
    }  
}
```

Run: SingleThreadExecutor x

"C:\Program Files\Java\jdk-17\bin\java.exe"

pool-1-thread-1执行线程任务 09:55:11.400713600

pool-1-thread-1执行线程任务 09:55:11.924764400

pool-1-thread-1执行线程任务 09:55:12.436082700

pool-1-thread-1执行线程任务 09:55:12.950352400

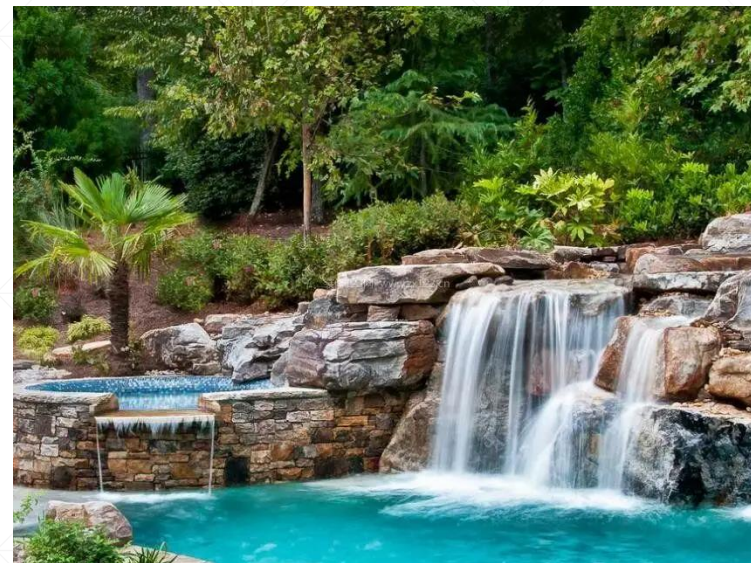
pool-1-thread-1执行线程任务 09:55:13.458387400

pool-1-thread-1执行线程任务 09:55:13.958987300

CachedThreadPool

这是一种可以缓存的线程池，是一种无界线程池，具有自动回收多余线程的功能

```
@NotNull
public static ExecutorService newCachedThreadPool() {
    return new ThreadPoolExecutor( corePoolSize: 0, Integer.MAX_VALUE,
                                   keepAliveTime: 60L, TimeUnit.SECONDS,
                                   new SynchronousQueue<Runnable>());
}
```



由于第2个参数maxPoolSize被设置为Integer.MAX_VALUE，这种线程池可能会创建非常多的线程，从而导致OOM。

测试代码



```
public class CachedThreadPool {  
    public static void main(String[] args) {  
        ExecutorService executorService = Executors.newCachedThreadPool();  
        for (int i = 0; i < 1000; i++) {  
            executorService.execute(new Task());  
        }  
    }  
}
```

Run: CachedThreadPool x

pool-1-thread-782执行线程任务 09:04:00.993085400
pool-1-thread-214执行线程任务 09:04:01.037769700
pool-1-thread-719执行线程任务 09:04:00.833115600
pool-1-thread-118执行线程任务 09:04:01.106875400
pool-1-thread-127执行线程任务 09:04:00.800757700
pool-1-thread-470执行线程任务 09:04:00.902918100
pool-1-thread-989执行线程任务 09:04:00.892946500

可以看到，为了执行1000个任务，CachedThreadPool创建了几百个线程。

直观了解CachedThreadPool工作原理-1

```
class ThreadTask implements Runnable {  
    2 usages  
    private Date initDate; //创建时间  
    5 usages  
    private String name;  
    1 usage  
    public ThreadTask(String name) {  
        initDate = new Date();  
        this.name = name;  
    }  
}
```

此类代表一个工作任务，执行时会输出其开始与结束的时间，并指明是由哪个线程执行的。

```
@Override  
public void run() {  
    var threadName :String = Thread.currentThread().getName();  
    System.out.printf("%s: 执行的任务 %s: 创建于: %s\n", threadName, name, initDate);  
    System.out.printf("%s: 执行的任务 %s: 开始于: %s\n", threadName, name, new Date());  
    try {  
        Long duration = (long) (Math.random() * 10);  
        System.out.printf("%s: 执行任务 %s: 花费时间 %d 秒\n", threadName, name, duration);  
        TimeUnit.SECONDS.sleep(duration);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    System.out.printf("%s: 执行的任务 %s: 结束于: %s\n", threadName, name, new Date());  
}
```

HowCachedThreadPoolWork.java

直观了解CachedThreadPool工作原理-2

```
//内部封装了一个线程池，负责执行计算任务，并输出线程池工作的状态信息
2 usages
class Server {
    5 usages
    private final ThreadPoolExecutor executor =
        (ThreadPoolExecutor) Executors.newCachedThreadPool();
    1 usage
    public void executeTask(ThreadTask task) {
        System.out.printf("Server: 来了一个新任务\n");
        //执行计算任务
        executor.execute(task);
        System.out.printf("Server:线程池大小: %d\n", executor.getPoolSize());
        System.out.printf("Server:激活线程数: %d\n", executor.getActiveCount());
        System.out.printf("Server:已完成任务: %d\n", executor.getCompletedTaskCount());
    }
    1 usage
    public void endServer() {
        executor.shutdown();
    }
}
```

捕获此刻线程池的状态信息

直观了解CachedThreadPool工作原理

```
public class HowCachedThreadPoolWork {  
    public static void main(String[] args) {  
        Server server = new Server();  
        //创建1000个任务, 提交给线程池执行  
        for (int i = 0; i < 1000; i++) {  
            ThreadTask task = new ThreadTask( name: "Task " + i);  
            server.executeTask(task);  
        }  
        server.endServer();  
    }  
}
```

从输出中可以看到, 线程池大约创建了900多个线程完成1000个任务.....

Run: HowCachedThreadPoolWork x

Server: 激活线程数: 901
Server: 已完成任务: 98
Server: 来了一个新任务
Server: 线程池大小: 902
Server: 激活线程数: 902
Server: 已完成任务: 98
pool-1-thread-902: 执行的任务 Task 999: 创建于: Sun Sep 18 14:55:58 CST 2022
pool-1-thread-902: 执行的任务 Task 999: 开始于: Sun Sep 18 14:55:58 CST 2022
pool-1-thread-902: 执行任务 Task 999: 花费时间 2 秒
pool-1-thread-5: 执行的任务 Task 8: 结束于: Sun Sep 18 14:55:59 CST 2022
pool-1-thread-42: 执行的任务 Task 49: 结束于: Sun Sep 18 14:55:59 CST 2022
pool-1-thread-33: 执行的任务 Task 39: 结束于: Sun Sep 18 14:55:59 CST 2022

任务的定时调用-1

使用ScheduledThreadPool实现延时和定时调用。

```
Run: ScheduledAtFixedRateDemo X
"C:\Program Files\Java\jdk-17\bin\java.exe"
Main: 开始于 15:40:49.239924200
Main: 再有 999 毫秒将会运行下一个定时任务
Main: 再有 479 毫秒将会运行下一个定时任务
Task: 运行: 15:40:50.250548900
Main: 再有 1964 毫秒将会运行下一个定时任务
Main: 再有 1464 毫秒将会运行下一个定时任务
Main: 再有 951 毫秒将会运行下一个定时任务
Main: 再有 436 毫秒将会运行下一个定时任务
Task: 运行: 15:40:52.259864
Main: 再有 1921 毫秒将会运行下一个定时任务
Main: 再有 1407 毫秒将会运行下一个定时任务
Main: 再有 893 毫秒将会运行下一个定时任务
Main: 再有 379 毫秒将会运行下一个定时任务
Task: 运行: 15:40:54.255858300
Main: 没有更多的任务: 15:40:54.381407400
Main: 结束于: 15:40:54.381407400

Process finished with exit code 0
```

示例在执行时，先等待1秒，之后按照2秒为间隔反复执行同一个任务，主线程则随机检查10次，然后关闭线程池并退出整个程序。

任务的定时调用-2

设置了10个“检查点”



```
public static void main(String[] args) {
    ScheduledExecutorService executor = Executors.newScheduledThreadPool( corePoolSize: 1);
    System.out.printf("Main: 开始于 %s\n", LocalTime.now());
    FixedRateTask task = new FixedRateTask( name: "Task");
    //1秒后, 每隔2秒运行任务
    ScheduledFuture<?> result = executor.scheduleAtFixedRate(task,
        initialDelay: 1, period: 2, TimeUnit.SECONDS);
    for (int i = 0; i < 10; i++) {
        System.out.printf("Main: 再有 %d 毫秒将会运行下一个定时任务\n",
            result.getDelay(TimeUnit.MILLISECONDS));
        try {
            TimeUnit.MILLISECONDS.sleep( timeout: 500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    //关闭线程池
    executor.shutdown();
    System.out.printf("Main: 没有更多的任务: %s\n", LocalTime.now());
    try {
        executor.awaitTermination( timeout: 5, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.printf("Main: 结束于: %s\n", LocalTime.now());
}
```

示例: ScheduledAtFixedRateDemo.java

任务的延时调用-1

延时调用：使用 `ScheduledExecutorService.schedule()` 方法



The screenshot shows a Java IDE's Run console for a project named 'ScheduleTaskDemo'. The console output is as follows:

```
Run: ScheduleTaskDemo x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
Main: 开始于 10:23:08.255988200
Task 0: 开始于 : 10:23:09.267900900
Task 1: 开始于 : 10:23:10.273276500
Task 2: 开始于 : 10:23:11.266226800
Task 3: 开始于 : 10:23:12.278175100
Task 4: 开始于 : 10:23:13.268536100
线程池于: 10:23:13.268536100 正式关闭。
Process finished with exit code 0
```

示例： `ScheduleTaskDemo.java`

示例以1秒为间隔，顺序启动了五个任务。

任务的延时调用-2

```
public class ScheduleTaskDemo {  
  
    public static void main(String[] args) {  
        var executor : ScheduledExecutorService = Executors.newScheduledThreadPool( corePoolSize: 1);  
        System.out.printf("Main: 开始于 %s\n", LocalDateTime.now());  
        // 执行5个任务  
        for (int i = 0; i < 5; i++) {  
            ScheduledTask task = new ScheduledTask( name: "Task " + i);  
            //实现延迟调用  
            executor.schedule(task, delay: i + 1, TimeUnit.SECONDS);  
        }  
        // 关闭线程池  
        executor.shutdown();  
        // 等待线程池的中止  
        try {  
            executor.awaitTermination( timeout: 1, TimeUnit.DAYS);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.printf("线程池于: %s 正式关闭。", LocalDateTime.now());  
    }  
}
```

常用的四种线程池构造参数

参数名	FixedThreadPool	CachedThreadPool	ScheduledThreadPool	SingleThreadPool
corePoolSize	构建时动态指定	0	构建时动态指定	1
maxPoolSize	同上	Integer.MAX_VALUE	Integer.MAX_VALUE	1
keepAliveTime	0秒	60秒	60秒	0秒
BlockingQueue	LinkedBlockingQueue	SynchronousQueue	DelayedWorkQueue	LinkedBlockingQueue



在实际开发中，推荐依据要处理任务的类型，选择合适的线程池类型。

小结: Executors的静态方法与线程池种类

Executors 类的方法	说明
<code>newSingleThreadExecutor</code>	单线程，如果出了异常死掉后，自动重新创建一个新的线程
<code>newFixedThreadPool</code>	固定数目的线程池
<code>newCachedThreadPool</code>	动态地按需求增加线程，重用已创建的线程，长时不用的线程被销毁
<code>newScheduledThreadPool</code>	固定数目，定期执行
<code>newSingleThreadScheduledExecutor</code>	单线程，定期执行
<code>newWorkStealingPool</code>	可以依据CPU核数创建多个任务队列，实现线程之间的工作负荷均衡。